

Antes de empezar !!

Necesitas solamente:

- VS Code o similar.
- Un navegador.
- Una carpeta con **index.html** y **main.js**
- Live Server es recomendable, aunque para muchos ejercicios bastará con abrir el HTML.
- No necesitas ninguna librería externa.



EJERCICIO 1 — Gestión de notas de una clase

Objetivo

Crear un programa que gestione las notas de una clase utilizando **variables**, **condicionales**, **bucles** y **funciones**.

No necesitas HTML complejo. Puedes hacer toda la interacción mediante `prompt()` y mostrar resultados con `console.log()` o `alert()`.

Enunciado

Crea un programa que permita gestionar las notas de un grupo de alumnos.

Primero pregunta cuántos alumnos hay en la clase.

Para cada alumno debes pedir:

- Nombre.
- Nota del examen.

La nota debe estar entre `0` y `10`.

Si el usuario introduce una nota incorrecta, debes volver a pedirla hasta que introduzca una válida.

Una vez introducidos todos los alumnos, el programa debe mostrar:

1. El número total de alumnos.
2. La nota media de la clase.
3. La nota más alta y quién la ha obtenido.
4. La nota más baja y quién la ha obtenido.
5. Cuántos alumnos han aprobado.
6. Cuántos alumnos han suspendido.
7. Cuántos han sacado:
 - Sobresaliente (`9-10`)
 - Notable (`7-8.99`)
 - Bien (`6-6.99`)
 - Suficiente (`5-5.99`)
 - Suspenso (`0-4.99`)
8. El porcentaje de aprobados.
9. El porcentaje de suspensos.

Condiciones

Debes crear **al menos estas funciones**:

- Una función para pedir una nota válida.
- Una función que determine si un alumno ha aprobado.
- Una función que calcule la media.
- Una función que determine la calificación correspondiente.

No puedes resolverlo haciendo absolutamente todo dentro de un único bloque de código.

EJERCICIO 2 — Cajero automático

Objetivo

Practicar **condicionales**, **switch**, **bucles**, **funciones**, **Math** y **variables**.

Enunciado

Simula un cajero automático.

El usuario empieza con un saldo inicial que tú establecerás en el programa.

El programa debe mostrar un menú:

1. Consultar saldo
2. Ingresar dinero
3. Retirar dinero
4. Consultar número de operaciones
5. Salir

El usuario podrá elegir opciones repetidamente hasta seleccionar "Salir".

Consultar saldo

Debe mostrar:

- Saldo actual.
- Número de operaciones realizadas.

Ingresar dinero

El usuario introduce una cantidad.

Debes comprobar que:

- Es un número.

- Es mayor que 0.

Si es correcta, se suma al saldo.

Retirar dinero

El usuario introduce una cantidad.

Debes comprobar:

- Que sea válida.
- Que sea mayor que 0.
- Que haya suficiente saldo.

No se puede retirar más dinero del disponible.

Número de operaciones

Debes llevar un contador de operaciones.

Por ejemplo:

- Ingreso → +1 operación.
- Retirada → +1 operación.
- Consulta de saldo → +1 operación.

Parte extra

Cuando el usuario salga del programa, muestra un resumen:

- Saldo final.
- Número de operaciones.
- Dinero ingresado durante la sesión.
- Dinero retirado durante la sesión.

Restricciones

Utiliza:

- `switch`.
 - Un bucle.
 - Funciones.
 - `Number`.
 - `Math` cuando tenga sentido.
-

EJERCICIO 3 — Tienda de videojuegos

Este ya es bastante más **tipo examen**.

Enunciado

Crea un programa que gestione una pequeña tienda de videojuegos.

Cada videojuego tendrá:

- Nombre.
- Precio.
- Género.
- Edad mínima.
- Stock.

Por ejemplo, tendrás que trabajar con varios videojuegos diferentes.

El programa debe permitir realizar las siguientes operaciones mediante un menú:

1. Mostrar todos los videojuegos
2. Buscar videojuego
3. Comprar videojuego
4. Añadir stock
5. Mostrar videojuegos baratos
6. Mostrar videojuegos sin stock
7. Mostrar estadísticas
8. Salir

1. Mostrar todos

Muestra todos los videojuegos disponibles indicando todos sus datos.

2. Buscar

El usuario introduce parte del nombre.

La búsqueda debe funcionar aunque escriba mayúsculas o minúsculas.

Por ejemplo, si existe:

The Legend of Zelda

debería poder encontrarlo buscando:

zelda

3. Comprar

El usuario introduce el nombre del videojuego y la cantidad que quiere comprar.

Comprueba:

- Que exista.
- Que haya suficiente stock.
- Que la cantidad sea válida.

Si la compra es correcta:

- Reduce el stock.
- Calcula el precio total.
- Si compra más de 3 unidades, aplica un descuento del 10%.

4. Añadir stock

Permite seleccionar un videojuego y aumentar su stock.

5. Videojuegos baratos

El usuario introduce un precio máximo.

Muestra todos los videojuegos cuyo precio sea inferior o igual a ese precio.

6. Sin stock

Muestra todos los videojuegos cuyo stock sea 0.

7. Estadísticas

Calcula:

- Precio medio.
- Videojuego más caro.
- Videojuego más barato.
- Número total de videojuegos.
- Stock total.
- Número de videojuegos sin stock.
- Género más repetido.

Restricciones

Aquí quiero que practiques especialmente:

- Arrays.
- Objetos.
- Funciones.
- Bucles.

- `find()`.
 - `filter()`.
 - `map()`.
 - `sort()`.
 - `includes()`.
 - `toLowerCase()`.
-

EJERCICIO 4 — Sistema de empleados

Enunciado

Crea un programa para gestionar empleados de una empresa.

Cada empleado tendrá:

nombre
edad
departamento
salario
antigüedad

Crea un array con varios empleados.

El programa tendrá este menú:

1. Mostrar empleados
2. Buscar empleado
3. Empleados de un departamento
4. Empleado con mayor salario
5. Empleado con menor salario
6. Salario medio
7. Subir salario
8. Ordenar empleados
9. Estadísticas
10. Salir

Buscar empleado

El usuario introduce un nombre.

La búsqueda no debe distinguir entre mayúsculas y minúsculas.

Departamento

El usuario introduce un departamento y se muestran sus empleados.

Mayor y menor salario

Debes localizar los empleados correspondientes sin ordenar innecesariamente todo el array.

Salario medio

Calcula el salario medio de todos los empleados.

Subir salario

El usuario selecciona un empleado y un porcentaje.

Ejemplo:

Empleado: Ana

Subida: 7%

Debes modificar su salario.

Ordenar

Permite ordenar por:

- Nombre.
- Edad.
- Salario.
- Antigüedad.

El usuario debe elegir el criterio.

Estadísticas

Mostrar:

- Salario medio.
- Edad media.
- Empleado con más antigüedad.
- Empleado con menos antigüedad.
- Número de empleados por departamento.

EJERCICIO 5 — Analizador de texto

Este es muy bueno para practicar `String`, `Array`, funciones y objetos.

Enunciado

Pide al usuario que introduzca un texto largo.

Por ejemplo, podría introducir un párrafo entero.

El programa debe analizarlo.

Debe calcular:

1. Número de caracteres.
2. Número de caracteres sin contar espacios.
3. Número de palabras.
4. Primera palabra.
5. Última palabra.
6. Palabra más larga.
7. Palabra más corta.
8. Número de veces que aparece cada palabra.
9. Palabra que aparece más veces.
10. Número de frases.
11. Número de vocales.
12. Número de consonantes.
13. Número de números que aparecen en el texto.

Importante

La búsqueda de palabras debe ser independiente de mayúsculas/minúsculas.

Por ejemplo:

JavaScript
javascript
JAVASCRIPT

deben considerarse la misma palabra.

Parte extra

Haz una función que reciba una palabra y diga cuántas veces aparece en el texto.

Otra función debe recibir una palabra y devolver cuántos caracteres tiene.

Otra debe recibir el texto y devolver un objeto con las estadísticas.

EJERCICIO 6 — Sistema de reservas de un cine

Este ya te lo pondría perfectamente como ejercicio largo de examen.

Enunciado

Crea un programa para gestionar las reservas de un cine.

Tendrás varias películas.

Cada película tendrá:

titulo
genero
duracion
edadMinima
precio

Además, tendrás varias sesiones.

Cada sesión tendrá:

pelicula
hora
sala
asientosTotales
asientosOcupados

El programa debe mostrar este menú:

1. Mostrar películas
2. Mostrar sesiones
3. Buscar película
4. Comprar entradas
5. Cancelar entradas
6. Mostrar sesiones disponibles
7. Estadísticas
8. Salir

Comprar entradas

El usuario selecciona una sesión.

Después introduce:

- Número de entradas.

- Edad del cliente.

Debes comprobar:

- Que exista la sesión.
- Que haya suficientes asientos.
- Que el cliente tenga edad suficiente para la película.
- Que el número de entradas sea válido.

El precio dependerá del número de entradas:

- 1 entrada → precio normal.
- 2-3 entradas → descuento del 5%.
- 4 o más → descuento del 10%.

Cancelar

El usuario selecciona una sesión y una cantidad de entradas.

Debes devolver esos asientos a la sesión.

Sesiones disponibles

Mostrar solamente sesiones que todavía tengan asientos libres.

Estadísticas

Calcular:

- Película con más entradas vendidas.
- Sesión con más ocupación.
- Dinero recaudado.
- Número total de entradas vendidas.
- Número de asientos libres.
- Porcentaje de ocupación de cada sesión.

EJERCICIO 7 — Gestión de una biblioteca

Este sería mi **ejercicio final de entrenamiento** antes de un examen.

Enunciado

Crea una aplicación para gestionar una biblioteca.

Cada libro tendrá:

título
autor
genero
año
disponible

Cada usuario tendrá:

nombre
edad
librosPrestados

El programa debe permitir:

1. Mostrar libros
2. Buscar libro
3. Mostrar libros de un autor
4. Mostrar libros por género
5. Prestar libro
6. Devolver libro
7. Registrar usuario
8. Mostrar usuarios
9. Mostrar estadísticas
10. Salir

Buscar

La búsqueda debe funcionar por:

- Título.
- Autor.
- Parte del título.

No debe distinguir mayúsculas/minúsculas.

Prestar

Para prestar un libro debes comprobar:

- Que el libro exista.
- Que esté disponible.
- Que el usuario exista.
- Que el usuario no tenga ya ese libro.
- Que el usuario no haya alcanzado el límite de préstamos.

Establece tú mismo el límite.

Devolver

Comprueba que:

- El usuario exista.
- Tenga ese libro prestado.

Después:

- El libro vuelve a estar disponible.
- Se elimina de los libros prestados del usuario.

Estadísticas

Calcula:

- Número total de libros.
- Libros disponibles.
- Libros prestados.
- Género con más libros.
- Autor con más libros.
- Usuario con más libros prestados.
- Año medio de publicación.

Restricción importante

Intenta hacerlo utilizando **funciones pequeñas**, por ejemplo:

```
buscarLibro()  
buscarUsuario()  
prestarLibro()  
devolverLibro()  
mostrarLibros()  
mostrarEstadisticas()
```

No hagas una función de 200 líneas.